

11. Eventual Consistency & Scaling Out: P2P-Systeme

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2024

11.1 Vorschau

- Begriffe
- Strukturierte P2P-Systeme -> Distributed Hash Tables
 - Chord
 - Pastry
 - CAN
- Unstrukturierte P2P-Systeme
 - Gnutella
- Zusammenfassung

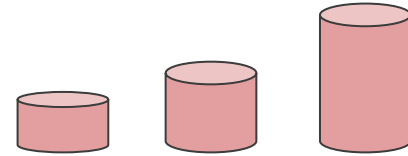


11.2 Begriffe



11.2 Begriffe

- Vertikale Skalierung (engl. scaling up)
 - Bestehender Server wird aufgerüstet
 - Bessere CPU, mehr RAM, schnellere SSD, ...
 - Problem: Server werden sehr teuer
- Horizontale Skalierung (engl. scaling out)
 - Es werden inkrementell Server hinzugefügt
 - Typischer Ansatz in der Cloud



11.2 Begriffe

- **Peer-to-Peer-Systeme** wurde entwickelt, um verteilte Ressourcen im Internet zu aggregieren. Primär wurden Dateien geteilt und Verfahren entwickelt, um diese schnell zu finden.
- Eigenschaften:
 - Dezentrale selbstorganisierende Architektur
 - Alle Rechner = Peers sind gleichberechtigt
 - Massive horizontale Skalierung (engl. scaling out)
 - Peers treten dynamisch bei und verlassen das System beliebig = Fluktuation (engl. churn)



11.2 Begriffe

- Client-Server:
 - Klassische Rollenverteilung, keine Interaktion zwischen Clients
 - Beispiele: WWW, Web-Services, ...
- Hybride P2P-Systeme:
 - Gemeinsame Nutzung von verteilten Ressourcen mit Interaktion zwischen Peers
 - Koordination durch zentrale Server
 - Beispiele: Napster, ICQ, ...
- Reine P2P-Systeme:
 - Vollständige dezentrale Organisation und Nutzung der Ressourcen



11.3 Strukturierte Peer-to-Peer Systeme



Allgemeines

- Bei strukturieren P2P-Systeme verbinden sich die Peers nicht willkürlich
- Stattdessen wird eine bestimmte Struktur erzeugt und aufrecht erhalten
 - Beispielsweise: Ring oder Torus
- Ziel ist es, dass Daten (im fehlerfreien Fall) schnell und immer gefunden werden, wenn sie im P2P-System gespeichert sind



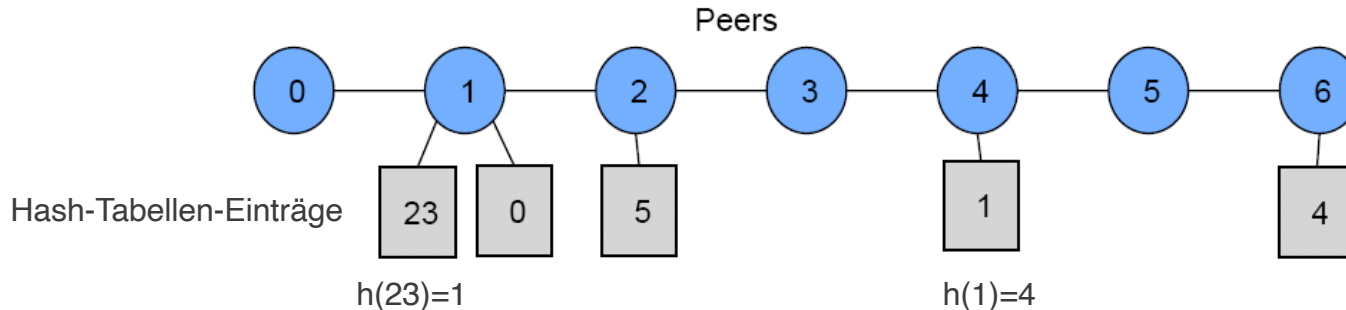
Hash-Tabelle

- Suche in $O(1)$
- Datenelemente haben einen Schlüssel K
- Hashfunktion h bildet Schlüssel auf Hashwert Q fester Länge ab (i. d. R. Zahlen bzw. Bitstrings) $h: K \rightarrow Q$
- Hashfunktionen haben die üblichen Eigenschaften ...



Verteilte Hash-Tabelle: naiver Ansatz

- Grundidee: Knoten als Zellen in einer Hashtabelle auffassen
 - Jedes Objekt hat einen Schlüssel.
 - Hash-Funktion h bestimmt den Speicherort für die Objekte
- Beispiel:
 - $h(x) = (3x+1 \bmod 23) \bmod 7$ ($7 = \text{Anzahl Peers}$)
 - Suche nach Objekt $o=1$: $\rightarrow$ berechne $h(1)=4$
 $\rightarrow$ Nachricht bis zu Knoten 4 weiterreichen



Verteilte Hash-Tabelle: naiver Ansatz

- Positiv bisher:
 - Es gibt eine eindeutige Zuordnung eines Objektes für einen Knoten. Diese kann jeder einfach anhand der Hash-Funktion lokal berechnen.
 - Gleichmäßige Verteilung der Daten auf die Peers
- Problem 1: Knotenankunft
 - Die Hashfunktion muss angepasst werden
 - Dadurch ist ein Re-Hashing aller Einträge notwendig
- Problem 2: Suche ist langsam $\rightarrow O(N)$
- Problem 3: Knotenausfall
 - Netzwerkkette unterbrochen
- Fazit: insgesamt keine gute Idee.



Verteilte Hash-Tabelle: Verbesserungsziele

- Effiziente Anpassung der Zuordnungen der Schlüssel bei Ausfall, Beitritt und Austritt von Peers
- **Effiziente Suche nach einem Schlüssel**
 - Peers durch Overlay-Netzwerk systematisch verbinden, um so ein effizientes Routing zu ermöglichen



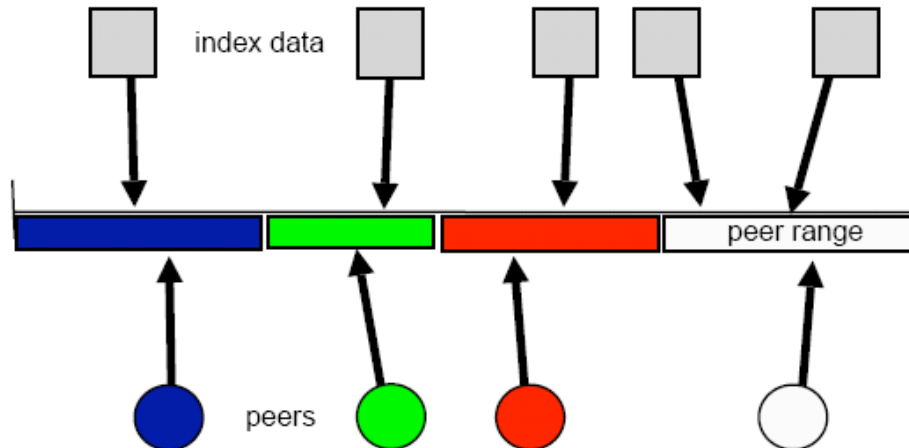
Verteilte Hash-Tabelle: Schlüssel-Indirektion

- Beim Beitritt zu einer Distributed Hash Table (DHT) wird einem Peer die Zuständigkeit für einen Teil des Schlüsselraums übertragen
- Wichtig: Das bedeutet nicht zwangsläufig, dass alle Daten auf diesen Peer übertragen werden müssen.
- Es reicht, wenn er eine Liste aller Peers hat, die die Daten bereithalten (Index)



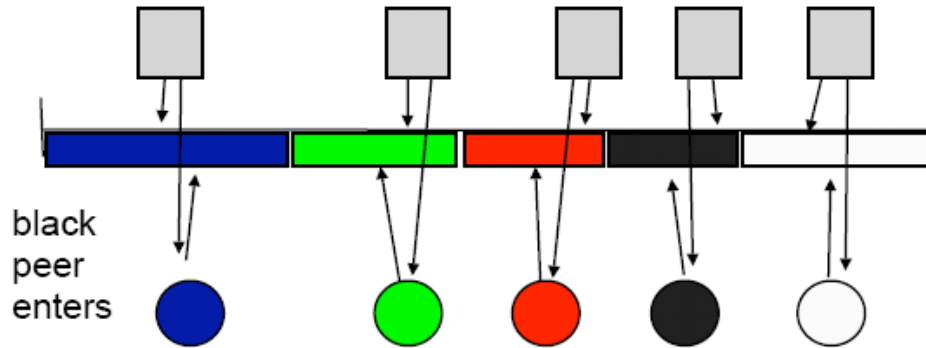
Verteilte Hash-Tabelle: Prinzip

- Jeder Peer ist für Teil des Hashwertebereichs zuständig
 - Peers werden an eine Position in einem linearen Wertebereich „gehasht“
 - Indexdaten werden ebenfalls per Hashfunktion auf diesen Wertebereich abgebildet und damit Knoten zugeordnet



Verteilte Hash-Tabelle: Prinzip

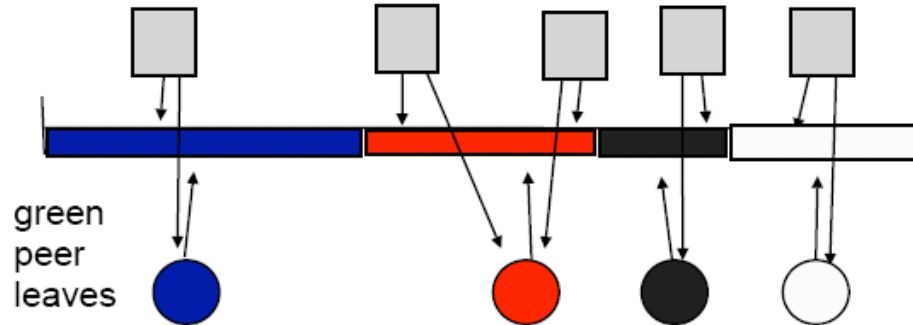
- Beitritt eines neuen **Peers**:
 - Zuteilung eines bestimmten Bereichs des Schlüsselraums und Migration der entsprechenden Indexdaten
 - Initialisierung mit Routinginformationen und Einbindung in die Routingstrukturen



Verteilte Hash-Tabelle: Prinzip

- Austritt eines Peers:

- Aufteilung des freiwerdenden Schlüsselraums auf benachbarte Knoten
- Migration der jeweiligen Indexdaten auf zuständige Knoten

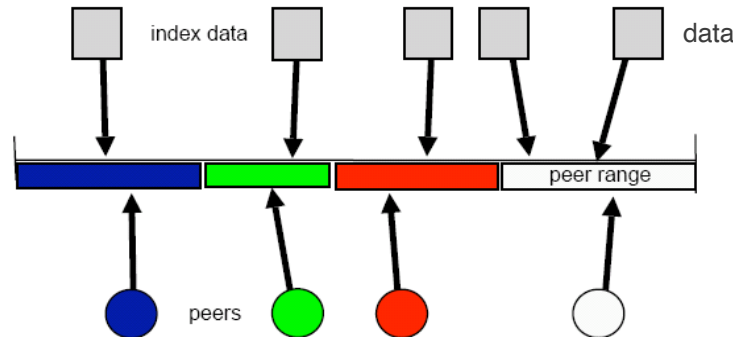


- Ausfall eines Knotens:

- Nutzung redundanter Routingwege und Replikation von Indexdaten + Daten
- Anpassung der Schlüsselraum-Zuweisungen

Distributed Hash Table (DHT)

- Basiert auf Consistent-Hashing (Karger, 1997)
 - Unterteilt den Hash-Wertebereich in nicht überlappende Teilbereiche
 - Für jeden Bereich ist ein Peer/Knoten zuständig
 - Minimiert die Anzahl der Neuordnungen, falls sich die Anzahl der Teilbereiche ändert (wenn sich die Anzahl an Peers ändert)



Chord



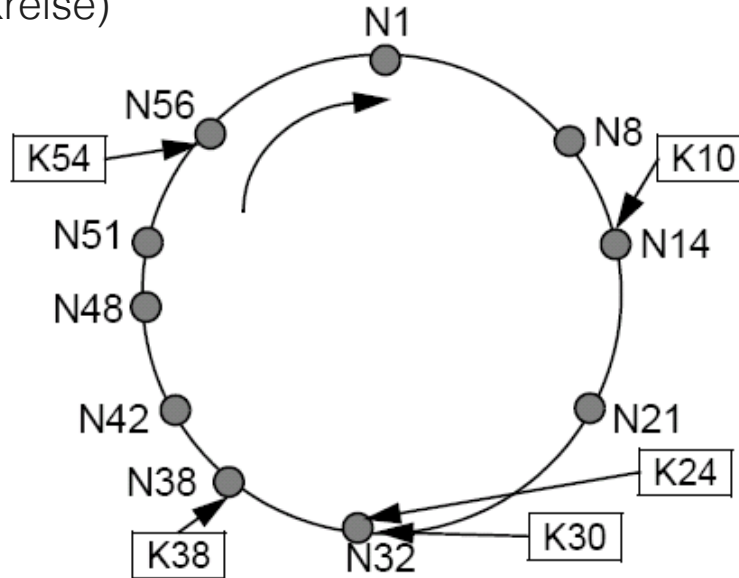
Chord: Grundidee

- Realisiert eine DHT
- Jeder Peer/Knoten besitzt eine eindeutige Hash-ID (z.B. per SHA-1)
- Ringförmige Anordnung der Peers gemäß Hash-IDs ($0..2^m-1$)
- Für jedes zu speichernde Objekt wird eine eigene Hash-ID erzeugt, verwaltet durch den Peer mit der unmittelbar im Ring folgenden Hash-ID (im Urzeigersinn)
 - Falls der Hash-Wertebereich klein ist, kann ein Objekt die gleiche Hash-ID wie ein Knoten haben. In diesem Fall verwaltet der Knoten mit der gleichen Hash-ID dieses Objekt.
- Hash-ID im Folgenden auch Key/Schlüssel genannt



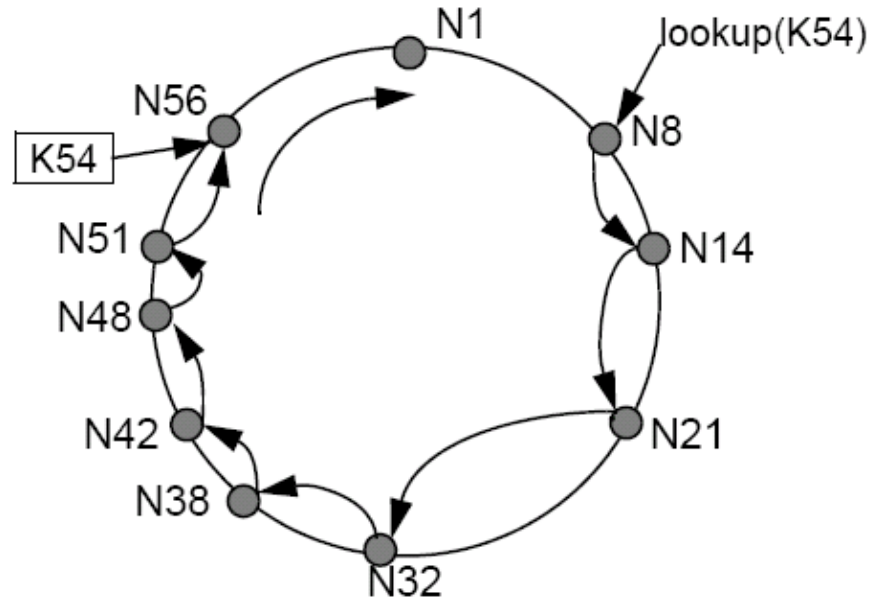
Chord-Grundidee

- Beispiel: #Knoten=10, Schlüssellänge $m=6 \rightarrow$ IDs = $\{0 \dots 63\}$
 - 5 verwaltete Objekte (Kästchen)
 - 10 Knoten (Kreise)



Naive Datensuche

- Jeder Knoten kennt seinen unmittelbaren Nachfolger (engl. successor) und leitet eine Anfrage solange weiter, bis der gesuchte Key zw. der eigenen ID und der des unmittelbaren Nachfolgers liegt.
- Dieser Nachfolger verwaltet den gesuchten Key.
- Komplexität $O(N)$



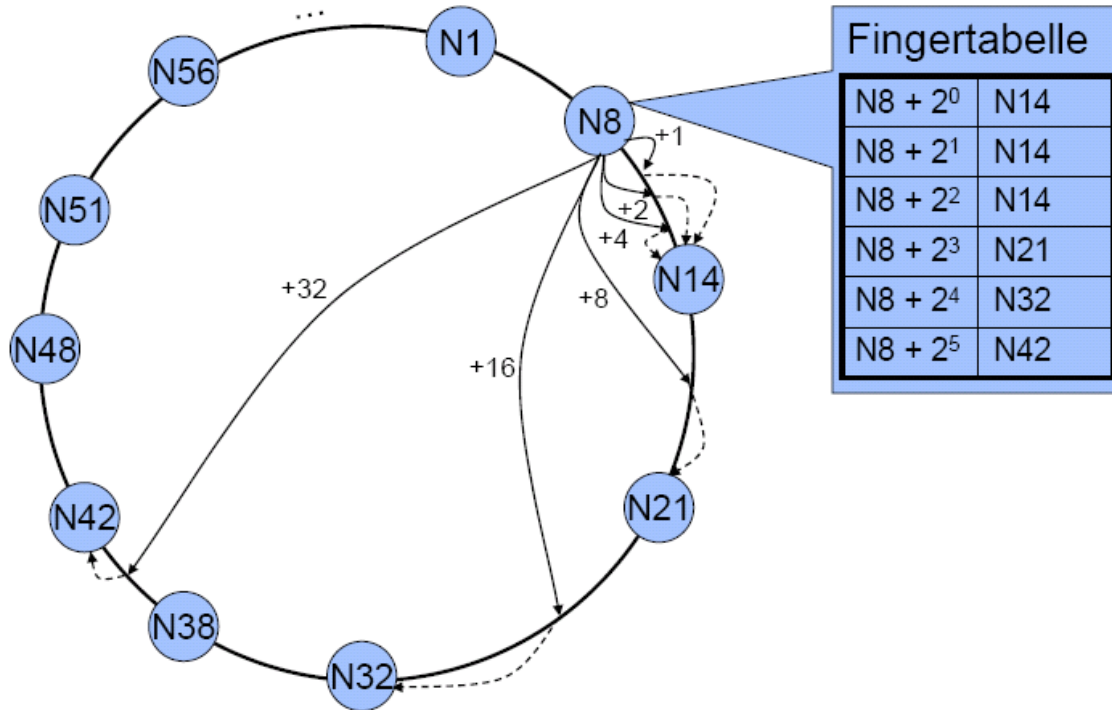
Effiziente Datensuche mithilfe der Fingertabelle

- **Fingertabelle** zur schnelleren Suche
 - Jeder Knoten kennt m Nachfolger (Tabelle hat m Einträge)
 - Distanz der Tabellen-Einträge im ID-Raum wächst exponentiell
 - Unmittelbarer Nachfolger im Ring ist der erste Eintrag in der Fingertabelle
- Eintrag in der Zeile k ist die ID des Knotens, der für die Hash-ID $= (n + 2^{k-1}) \bmod 2^m$ zuständig ist
- $1 \leq k \leq m$ (n =eigene ID, m =Schlüssellänge)
- Ermöglicht Suche mit $O(\log N)$



Effiziente Datensuche mithilfe der Fingertabelle

- Beispiel: Fingertabelle des Knotens N8 (Schlüssellänge: $m=6$)

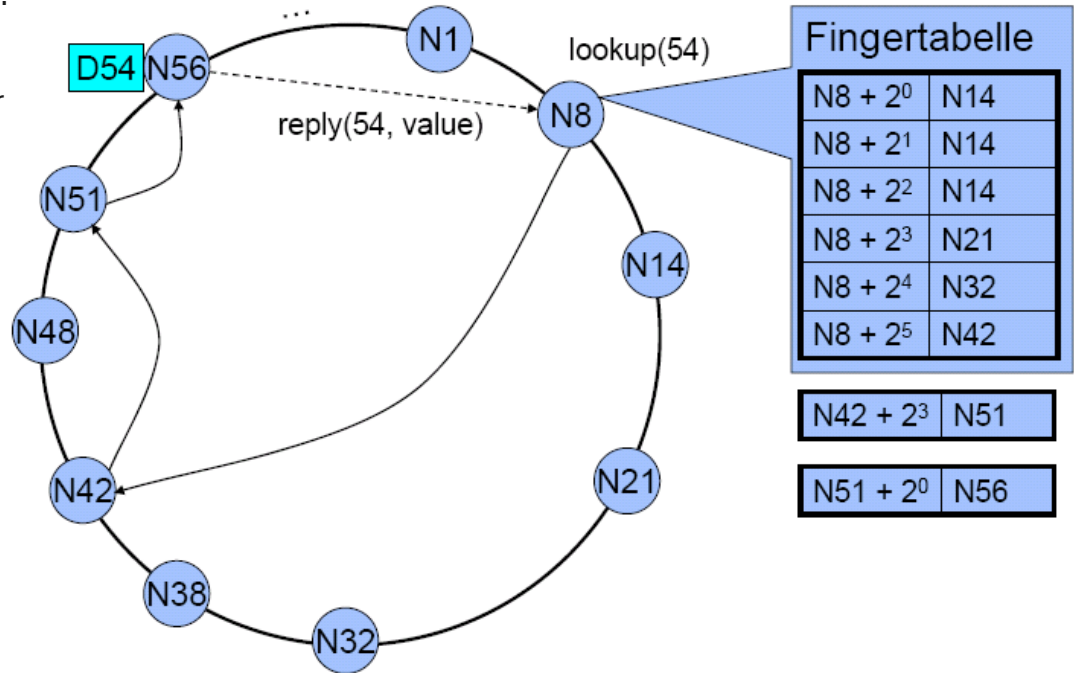


Effiziente Datensuche mithilfe der Fingertabelle

- Suche nach einem Schlüssel mithilfe der Fingertabelle

- Prinzip auf jedem Knoten:

- Schaue zuerst, ob der unmittelbare Nachfolger zuständig ist
- Ansonsten nehme den Eintrag dessen ID numerisch am nächsten zum gesuchten Schlüssel, aber kleiner wie dieser ist



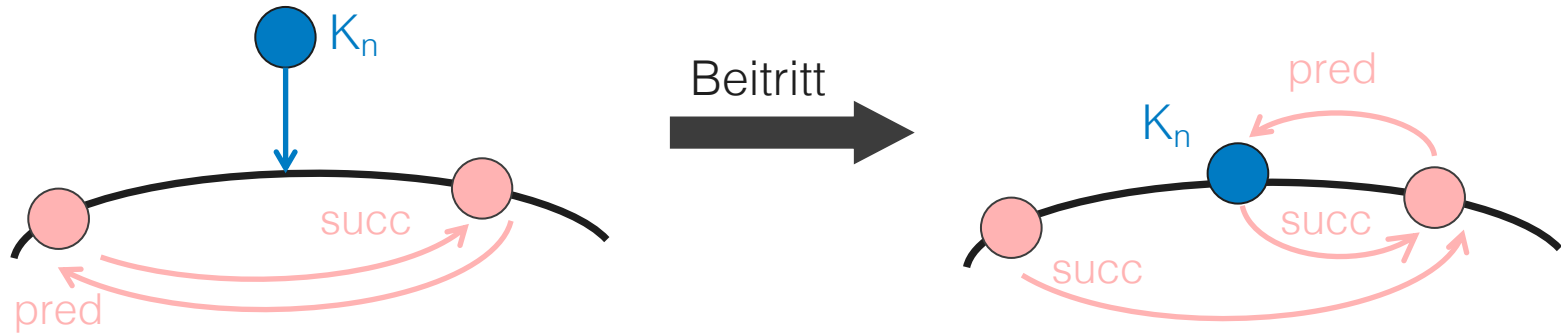
Beitritt eines neuen Knoten

- Knoten K_n will beitreten und berechnet zunächst seine Hash-ID
- K_n sucht seine eigene ID, Einstieg ist K_o (beliebiger Knoten, den er kennt)
 - Damit ermittelt K_n seinen Nachfolger $K_s = \text{succ}(K_n)$
 - Diesen kontaktiert er, um sich als neuen Vorgänger $\text{pred}(K_s)$ zu registrieren
 - Alle Schlüssel, für die K_n nun zuständig ist ($\text{keys} \in [\text{pred}(n), n]$) werden verschoben vom $\text{succ}(K_n)$ zu K_n
- Nun baut K_n seine Fingertabelle auf:
 - Hierzu sucht er die IDs der Einträge für seine Tabelle über K_s



Beitritt eines neuen Knoten (2)

- Ergebnis:
 - Nun hat K_n einen gültigen Nachfolgerzeiger und eine aufgebaute Fingertable
 - Ferner ist er seinem Nachfolger bekannt, nicht jedoch seinem Vorgänger
→ „Stabilization Protocol“ (siehe später)
 - Ferner ist K_n nicht in den Fingertables der Anderen berücksichtigt
→ „fix_fingers()“ (siehe später)



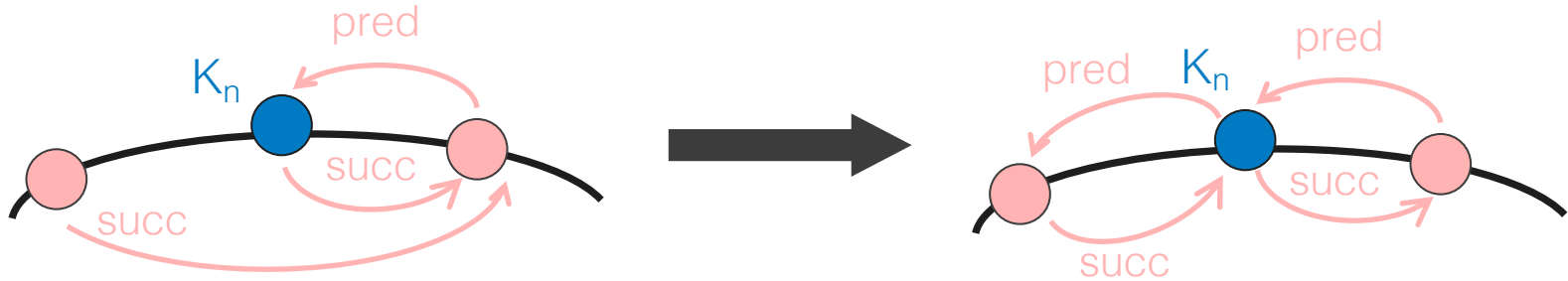
Beitritt eines neuen Knoten (2)

- Anfragen erreichen nun auch K_n , aber nicht direkt, sondern derzeit nur über seinen Nachfolger.
- Grund: K_n ist nicht in den Fingertables der anderen Knoten bekannt $\rightarrow$ „fix_fingers()“



Stabilization Protocol

- Wird periodisch von allen Knoten ausgeführt.
- Zur Validierung und Aktualisierung der Vorgänger- und Nachfolgerzeiger.
- Wichtig für Knotenbei- und austritt sowie im Fehlerfall, damit der Ring wieder repariert wird
- Beispiel: Vorgänge von K_n erkennt seinen neuen Nachfolger K_n



Stabilization Protocol: Ablauf für Knoten K_i

- K_i fragt seinen Nachfolger $K_s = \text{succ}(K_i)$ nach dessen Vorgänger $K_p = \text{pred}(K_s)$
- Falls $K_i = K_p$
 - Beide Knoten sind sich einig, dass sie unmittelbar aufeinander im Ring liegen
- Falls K_p zw. K_i und K_s liegt
 - K_p ist kürzlich beigetreten und ist der neue Nachfolger von K_i
 - K_i aktualisiert seinen Nachfolgezeiger auf K_p
- K_i informiert seinen Nachfolger (K_s oder K_p), dass er der Vorgänger ist



Aktualisierung der Fingertabelle

- Da veraltete Fingertabellen die Datensuche nur minimal verlangsamen, werden sie verzögert aktualisiert (→ lazy update).
- Jeder Knoten K_n führt periodisch eine „fix_fingers()“-Funktion aus:
 - Diese iteriert über die Einträge der Fingertable
 - Und sucht für jeden Eintrag i den aktuell gültigen Nachfolger von $n + 2^{i-1}$



Abmelden eines Knotens

- Knoten benachrichtigt seinen Vorgänger und Nachfolger und übergibt seine Schlüssel an seinen Nachfolger.
- Durch das ordnungsgemäße Abmelden bleibt die Ringstruktur erhalten.



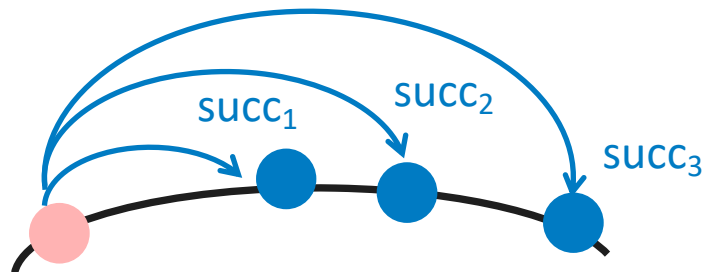
Ausfall eines Knotens

- Zunächst Fingertabelle verwenden, um den Ring zu reparieren
- Wenn ein Peer feststellt, dass sein Nachfolger nicht mehr erreichbar ist, setzt er den Nachfolger auf den nächstgelegenen erreichbaren Finger
 - evt. Werden dadurch Peers „übersehen“
- Stabilization Protocol stellt dann die richtige Ringstruktur wieder her (evt. in mehreren Schritten).
- Problem: falls alle Peers in der Finger-Table gleichzeitig ausfallen. (Dies ist allerdings sehr unwahrscheinlich.)



Verbesserte Zuverlässigkeit

- Jeder Knoten hält eine kurze Liste der Länge r und speichert darin r Nachfolger statt nur einem einzigen
- Damit können $r-1$ simultane Knotenausfälle maskiert werden
- Jeder Knoten repliziert die Indexdaten aller Knoten in seiner Nachfolgerliste
- Merkt ein Knoten dass sein Nachfolger ausgefallen ist, so ersetzt er diesen durch einen anderen aus seiner Liste.



Lastverteilung

- Simulationen bei einer Netzwerkgröße von 10^4 Knoten und der Abbildung von 10^5 bis 10^6 zufälligen Schlüsseln auf einen Chord-Ring ergeben eine ungleichmäßige Verteilung der Schlüssel.
 - Es gibt Knoten, die keine Schlüssel zugeteilt bekommen, andere hingegen verwalten sehr viele Schlüssel.
- Lösung: Einführung mehrerer virtueller IDs pro Knoten
- Bem.: Zusätzlich können durch die Anzahl der verwalteten virtuellen IDs pro Knoten Leistungsunterschiede zwischen verschiedenen Knoten berücksichtigt werden.



Literatur

- I. Stoica et.al., „Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications“, ACM SIGCOMM, San Deigo, 2001.
- Weitere Infos: <https://github.com/sit/dht>



Pastry



- Pastry ist ebenfalls ein strukturiertes P2P-System:
 - Projektseite: <http://research.microsoft.com/~antr/Pastry>
 - "Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems". A. Rowstron and P. Druschel, Int. Conference on Distributed Systems Platforms, Germany, November, 2001
- In Pastry ist die numerische Nähe von gesuchten Keys zum jeweiligen Knoten wichtig.



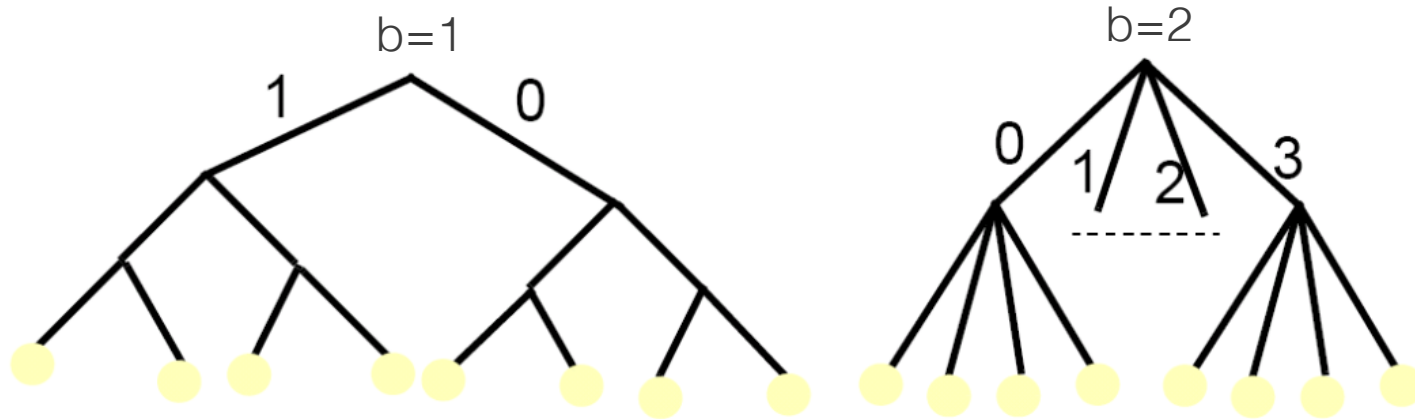
Allgemeines

- Knoten und Daten werden mit einer eindeutigen ID versehen.
- IDs werden als Zahlenfolge zur Basis 2^b (meist $b = 4 \rightarrow$ Hexadezimal) betrachtet.
- Meist werden 128 Bit breite IDs verwendet
- Knoten = node-IDs; Daten = keys.
- **Zuordnung: Ein Key (und damit die Daten) befindet sich auf dem Knoten, dessen node-ID numerisch am nächsten zu einem Key ist.**



Allgemeines

- In Pastry werden Peers als Blätter eines Baumes mit dem Verzweigungsgrad 2^b abgebildet:



Routing: Grundprinzip

- 1) Ein Knoten leitet eine Nachricht an einen anderen Knoten weiter, dessen Präfix mind. um eine Stelle mehr mit dem Key der Nachricht übereinstimmt als die eigene node-ID.
- 2) Kennt ein Knoten keinen anderen Knoten mit einer größeren Präfixübereinstimmung, wird die Nachricht an einen Knoten weitergeleitet, dessen node-ID numerisch näher zu dem Key der Nachricht ist als die eigene node-ID.

Gesuchter Knoten:023104

Start	1.Hop	2.Hop	3.Hop	4.Hop	5.Hop	Ziel
332401	032101	022100	023203	023113	023102	023104



Routing-Informationen (auf jedem Peer)

- **Routing-Tabelle:** speichert Verweise in den ID-Raum
 - Vgl. Finger-Tabelle in Chord
- **Leaf-Set:** speichert Knoten die im ID-Raum sehr nahe sind.
 - vgl. Chord Successor-List
- **Neighborhood-Set:** speichert Knoten die topologisch nahe sind
 - Nähe ermittelt mithilfe von zum Beispiel Anzahl IP-Hops oder Ping-Round-Trip-Time (RTT)



Routingtabelle des Knoten i

- Tabelle zur präfixbasierten Zustellung von Nachrichten
- $\lceil \log_{2^b} N \rceil$ Zeilen, wobei N die Knotenanzahl ist
- $2^b - 1$ Einträge pro Zeile.
 - b = Exponent des Verzweigungsgrades
- Einträge in der Zeile k sind Knoten deren Präfix mit dem von i in k Stellen übereinstimmen.
 - Knoten die keinen gemeinsamen Präfix mit i haben sind in der 0-ten Zeile gelistet



Routingtabelle des Knoten i

- Für einen Eintrag der Routingtabelle kann es keinen oder evt. viele passende node-IDs geben, wobei diejenigen verwendet werden, die topologisch nahe zu Knoten i sind
 - Siehe später
- Routingtabelle ist i.d.R. nicht voll befüllt, da ID-Raum nicht voll belegt ist



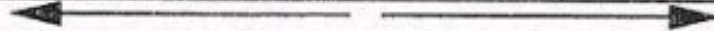
Beispiel: Routingtabelle des Knoten 103220

- Basis $b=2$
 - Verzweigungsgrad = 4
- 3 Einträgen pro Zeile
 - Eigene Präfixstelle in jeder Zeile jeweils ausgelassen

0	031120		201303	312201
1		110003	120132	132012
2	100221	101203	102303	
3		103112		103302
4		103210		
5				

Leaf Set

103123	103210	103302	103330
--------	--------	--------	--------

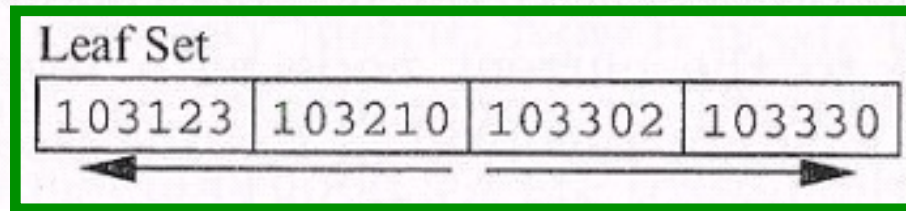


Neighborhood Set

031120	312201	120132	101203
--------	--------	--------	--------

Beispiel: Leaf-Set des Knoten 103220

- Menge der numerisch nächsten (bekannten) node-IDs zur eigenen node-ID (= Blätter des Baumes).
- Enthält L Einträge, wobei L meist 2^b entspricht.
- Davon sind $L/2$ kleiner als die eigene node-ID und $L/2$ größer.
- Beispiel: node-ID = 103220 (hier 4 Einträge, für $b=2$)



Beispiel: Neighborhood-Set des Knoten 103220

- Liste von topologisch nahen Nachbarn.
 - Ermittelt mit einer Heuristik, z.B. IP-Hops, Ping Round Trip Time, ...
- Dient der Optimierung der Routingtabelleneinträge hinsichtlich Lokalität und wird nur im Ausnahmefall für das Routing verwendet
- Meist 2^b Einträge $\rightarrow$ im Beispiel $2^2 = 4$
- Beispiel:

Neighborhood Set			
031120	312201	120132	101203

Routingvorgang für eine Nachricht mit dem Key k

- 1) Falls k im Wertebereich des Leaf-Set liegt, wird die Nachricht an Knoten mit der geringsten numerischen Distanz versendet.
- 2) Ansonsten ermittle die Länge des gemeinsamen Präfixes der node-ID und dem Key k . Falls es einen Eintrag in der Routingtabelle gibt, dessen Präfix um mindestens eine Stelle länger ist, wird die Nachricht an den diesen Knoten versendet.
- 3) Sollte weder 1 noch 2 erfüllt sein, wird die Nachricht an den numerisch nächsten Knoten aus der Vereinigungsmenge von Leaf-Set, Routingtabelle und Neighborhood-Set gesendet (tritt selten ein)



Beispiel: Routing nach Fall 1: Suche Key=103200

- Tabelle des Knoten mit node-ID = 103220

0	031120		201303	312201
1		110003	120132	132012
2	100221	101203	102303	
3		103112		103302
4		103210		
5				

Leaf Set

103123	103210	103302	103330
--------	--------	--------	--------

← →

- Gesuchter Key=103200 liegt im Wertebereich des Leaf-Sets: $103123 < 103200 < 103210$
- Numerisch nächste ID dieser beiden Einträge ist 103210
- Weiterleiten an den Knoten 103210 aus dem Leaf-Set



Beispiel: Routing nach Fall 2

- Suche key=102022 (eigene node-ID=103220)

- (1) Nicht im Wertebereich des Leaf-Sets
→ Routingtabelle konsultieren
- (2) Obwohl 101203 numerisch näher ist, wird diese Anfrage an **102303** weitergeleitet, da dieser Knoten den längsten Präfix 102 gemeinsam hat (und 101203 nur 10)

Routing Table

0	031120		201303	312201
1		110003	120132	132012
2	100221	101203	102303	
3		103112		103302
4		103210		
5				

Leaf Set

103123	103210	103302	103330
--------	--------	--------	--------

Neighborhood Set

031120	312201	120132	101203
--------	--------	--------	--------

Beispiel: Routing nach Fall 3

- Suche Key=103000 (eigene node-ID=103220)
 - Nicht im Wertebereich des Leaf-Sets
 - Routing Tabelle enthält keinen Eintrag mit längerem Präfix als der aktuelle Knoten
 - Anfrage wird zu **103112** weitergeleitet, der den gleichen Präfix 103 wie der aktuelle Knoten hat, aber numerisch am nächsten ist.
- | Routing Tabelle | |
|-----------------|----|
| 0 | 03 |
| 1 | |
| 2 | 10 |
| 3 | |
| 4 | |
| 5 | |

0	031120		201303	312201
1		110003	120132	132012
2	100221	101203	102303	
3		103112		103302
4		103210		
5				

Leaf Set

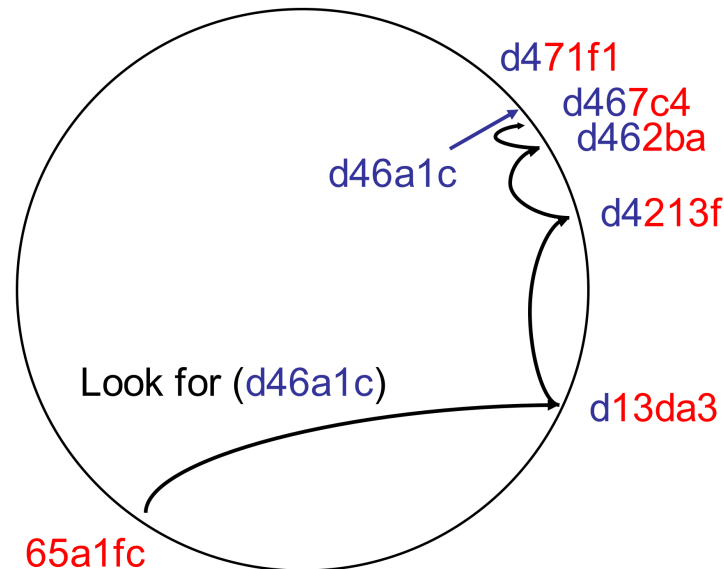
103123	103210	103302	103330
--------	--------	--------	--------

←-----→

Neighborhood Set			
031120	312201	120132	101203

Beispiel: Routing (Key-Space graph. als Ring dargestellt)

- Gesucht wird Key: **d46a1c**, ausgehend von node-ID **65a1fc**
 - Routing leitet Nachricht zu dem Knoten dessen ID der gesuchten am nächsten ist (hier d467c4)
 - Mit Prefix-Routing wird zunächst in die Nähe des Ziels gesprungen
 - Die genaue Positionierung erfolgt später über das Leaf-Set
 - Routing erfolgt in $O(\log N)$
 - Bem.: Mit jedem Schritt reduziert sich die Anzahl der verbleibenden Knoten exponentiell. Durch das prefixbasierte Routing.



Integration eines neuen Knoten (1)

- Ein neuer Knoten K_n erzeugt selbst seine node-ID (mit der bekannten Hashfkt.)
- Kontaktiert dann einen **bereits im Netzwerk aktiven Knoten K_a**
 - K_a soll topologisch nahe zu K_n sein
 - K_a wird über eine spezielle Join-Nachricht kontaktiert, die als Zieladresse die node-ID von K_n hat
- **Neighborhood-Set: übernimmt K_n von K_a**
 - Dies ist sinnvoll, da K_a topologisch nahe zu K_n ist
 - Somit sind auch die Knoten im Neighborhood-Set von K_a topologisch nahe zu K_n



Integration eines neuen Knoten (2)

- **Leaf-Set:** K_n schickt eine Join-Nachricht mit seiner ID über K_a
 - Nachricht erreicht entsprechend dem Routingalgorithmus K_c
 - Knoten K_c hat die numerisch nächste node-ID zu K_n
→ K_n übernimmt den Leaf-Set von K_c
- **Routing-Table:** alle Knoten die die Join-Nachricht zu K_c weiterleiten senden an K_n ihre Routing-Tabelle.
 - Auswahl der Einträge siehe folgende Seiten
 - Abschließend übermittelt K_n seine Routinginformationen an alle ihm dann bekannten Knoten damit diese ihre Einträge bzgl. topologischer Nähe optimieren können.
- Bem.: Einträge werden aktiv und direkt abgeglichen im Gegensatz zu Chord.



Lokalität in der Routingtabelle (1)

- Proximity-Space = geographischer Raum
- ID-Space = logischer ID-Raum
- Ziel: Jeder Eintrag in der Routing-Tabelle verweist auf einen Knoten der geographisch nahe ist, mit dem entsprechenden Präfix.



Lokalität in der Routingtabelle (2)

- Beitritt von Knoten K_n : Join-Nachricht wandert von $K_a \rightarrow K_b \rightarrow K_c \rightarrow \dots$
- **Knoten in Zeile 0 der Routingtabelle von K_a** sind nahe zu K_a
→ Da K_n nahe zu K_a ist, sind dies gute Kandidaten für Zeile 0 von K_n



Lokalität in der Routingtabelle (3)

- Beitritt von Knoten K_n : Join-Nachricht wandert von $K_a \rightarrow K_b \rightarrow K_c \rightarrow \dots$
- Knoten in Zeile 1 von K_b sind geeignet für Zeile 1 von K_n
 - Knoten in Zeile 1 von Knoten K_b sind relativ nahe zu K_b
Da K_b vergleichsweise nahe zu K_a ist, sind die
Einträge in Zeile 1 auch relativ nahe zu K_n



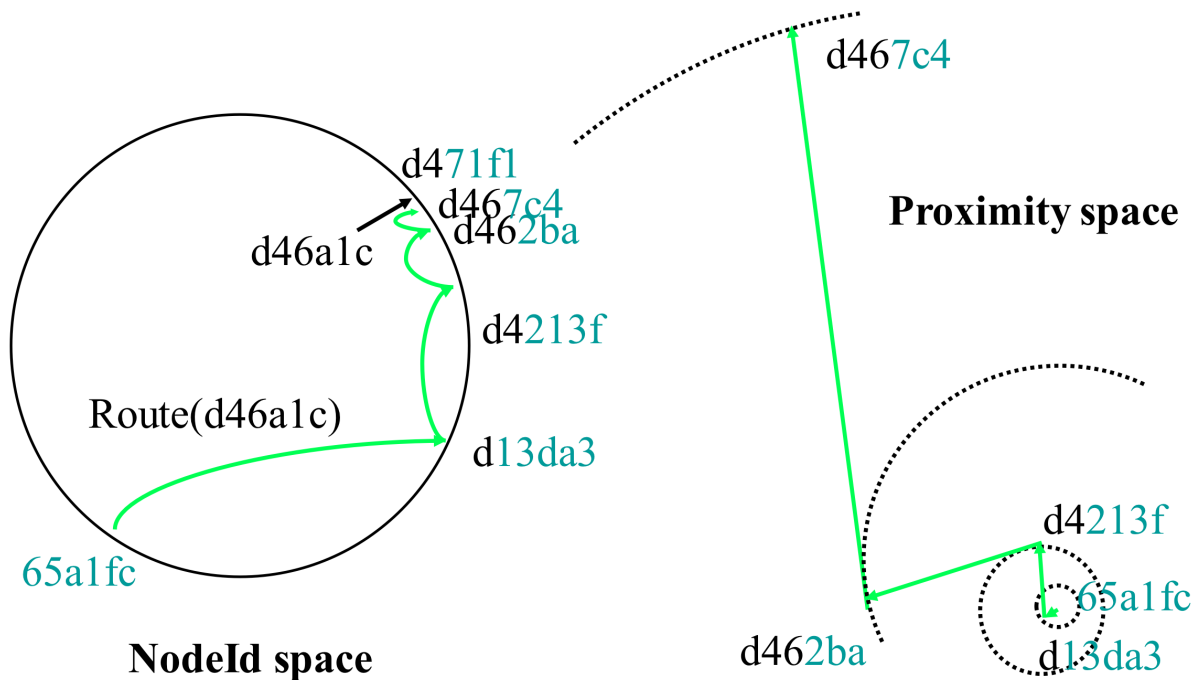
Lokalität in der Routingtabelle (4)

- Beitritt von Knoten K_n : Join-Nachricht wandert von $K_a \rightarrow K_b \rightarrow K_c \rightarrow \dots$
- Knoten in Zeile 2 von K_c sind geeignet für Zeile 2 von K_n
 - Usw.
 - Ergibt gute Näherung für alle Einträge bzgl. geographischer Distanz
 - Warum ist dies so?
 - Einträge jeder nachfolgenden Zeile stammen aus der exponentiell schrumpfenden Menge von Knoten
 - Je größer gemeinsamer Präfix ist, desto wahrscheinlicher wird ein größerer geographischer Abstand (da es weniger Auswahl an Einträgen gibt)



Lokalität in der Routingtabelle (5)

- Routing: Vergleich NodeID-Space und Proximity-Space
 - Proximity-Distanz wächst exponentiell bei jedem Schritt



Lokalität in der Routingtabelle (5)

- Zusätzlich erfolgt noch ein weiterer Schritt
 - K_n fordert nach dem Beitritt die Routing- und Neighborhood-Sets von allen Knoten an, die er in seinen Tabellen eingetragen hat (ausgenommen Leaf-Set)
 - Optimiert mit diesen Infos seine Einträge nochmals hinsichtlich der geographischen Distanz



Ausfall eines Knotens des Leaf-Set

- Wird erst beim Versuch eine Nachricht zuzustellen erkannt.
- In diesem Fall wird der zur eigenen ID numerisch entfernteste Knoten aus dem eigenen Leaf-Set kontaktiert und zwar auf der Seite des Leaf-Sets in der der ausgefallene Knoten liegt
 - Ist dieser nicht erreichbar werden notfalls andere Einträge aus dem eigenen Leaf-Set verwendet
 - Wenn alle Einträge im Leaf-Set nicht erreichbar sind, muss die eigene node-ID erneut gesucht werden und damit der Leaf-Seat aktualisiert werden
- Der angefragte Knoten übermittelt sein Leaf-Set und der fehlende Eintrag wird damit ersetzt.



Ausfall eines Knotens innerhalb der Routing-Tabelle

- Wird auch erst bei der Nachrichtenzustellung erkannt.
- In diesem Fall wird ein Knoten der gleichen Tabellenzeile angefragt.
- Dieser sendet die entsprechende Zeile seiner Routingtabelle und der ausgefallene Knoten wird ersetzt.

Ausfall eines Knotens des Neighborhood-Set

- Wird nur sehr selten beim Routing angesprochen.
- Deswegen werden diese Einträge periodisch überprüft.
- Antwortet ein Knoten nicht, werden die anderen Knoten des Neighborhood-Sets benachrichtigt und die Neighborhood-Sets untereinander ausgetauscht und abgeglichen.



Bemerkungen (1)

- Routing bei N Knoten mit $O(\log N)$ möglich.
- Basis b und Anzahl der Knoten entscheiden über die Größe der Routing-Tabelle und maximale Anzahl an Hops der Route zwischen zwei Knoten
- Zahlenbeispiele für $b=4$
 - Bei 10^6 Knoten im Mittel 75 Einträge in der Routing-Tabelle und eine maximale Such-Weglänge von 5
 - Bei 10^9 Knoten im Mittel 105 Einträge in der Routing-Tabelle und eine maximale Such-Weglänge von 7



Bemerkungen (2)

- Mit Pastry realisierte Anwendungen (Auszug):
 - Scribe - Gruppenkommunikation und Event-Dienst
 - Past - verteiltes Dateisystem
 - Squirrel - kooperativer Web-Cache
 - Post - Messaging-Dienst
- Alle Anwendungen auch von Microsoft Research

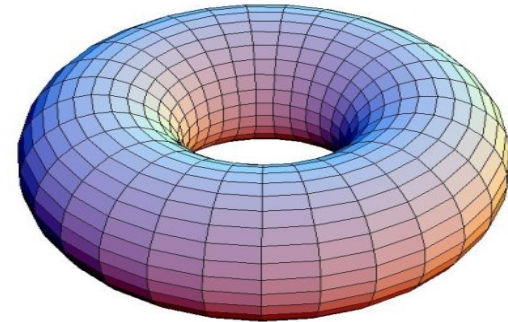


Content Addressable Network (CAN)



Allgemeines

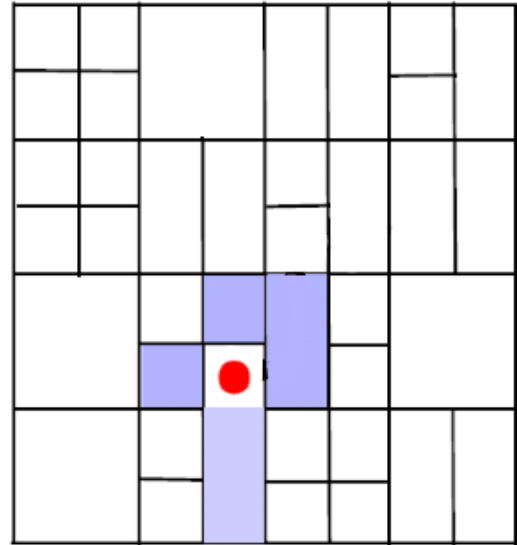
- Jeder Knoten verwaltet einen Teil des Koordinatenraums
- Zuständigkeitsbereiche der Knoten überlappen sich nicht und es gibt keine Lücken.
- Organisation der Knoten in einem d-dimensionalen, virtuellen, kartesischen Koordinatensystem auf einem Torus (Wraparound) →



- "A Scalable Content-Addressable Network",
Sylvia Ratnasamy et.al., SIGCOMM'01, 2001.

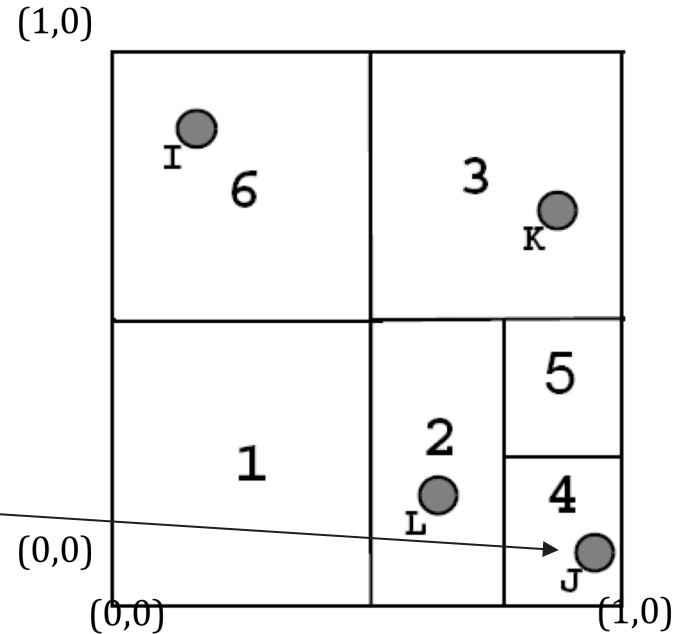
Allgemeines

- Hashfunktion bildet Key auf einen **Punkt im Raum** ab
- Daten zu einem Key werden von einem Knoten gespeichert, zu dessen Bereich der Punkt gehört.
- **Angrenzende Bereiche heißen Nachbarn** →
 - Dies sind Knoten deren Rechtecke sich in einer Dimension überlappen



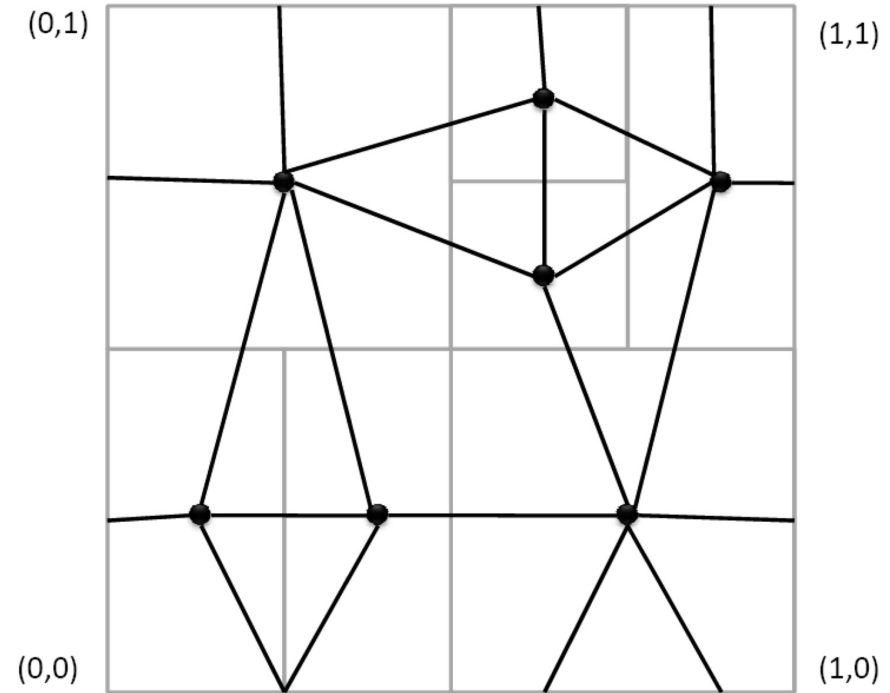
Beispiel für 2D-Koordinatensystem

- Nachbarn von Knoten 6: 3, 1
- Nachbarn von Knoten 2: 3, 1, 5, 4
- Knoten 1 verwaltet den Bereich:
 $x=[0;0,5]$ $y=[0;0,5]$.
- Schlüssel $J=(0,8; 0,1)$ —————
- Bem.: 2 D Hashfunktion
 - Verwende 160 Bit Hashfunktion –



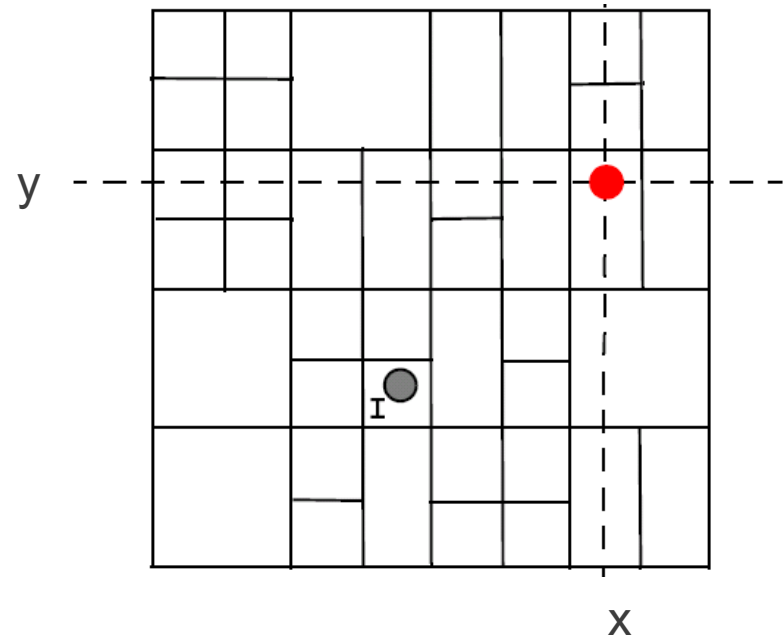
Verbindungsstruktur

- Verbindungen zwischen Nachbarn →
- Wrap-Around-Kanten
→ Torus



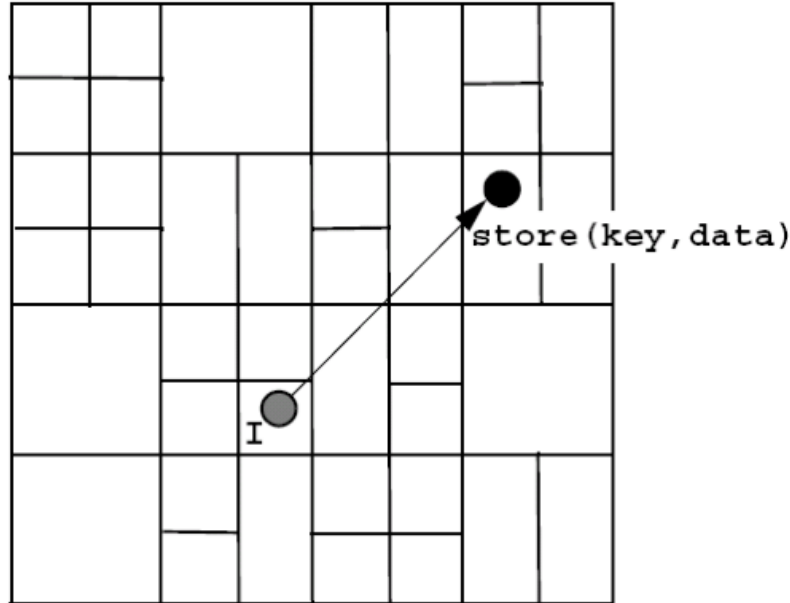
Einfügen / Abfragen von Daten

- Befehl: insert(key, data) / retrieve(key)
- Schritt-1: Bestimmung des **Abbildungspunkts des Keys** im Koordinatenraum:
 $v = h(\text{key})$
 $x = \text{obere 80 Bits von } v$
 $y = \text{untere 80 Bits von } v$



Einfügen / Abfragen von Daten

- Schritt-2: Transfer der Daten/Key zum Abbildungspunkt (x, y)
- Schritt-3: Speichern der Daten durch den zuständigen Knoten / Transfer der Daten zum anfragenden Knoten



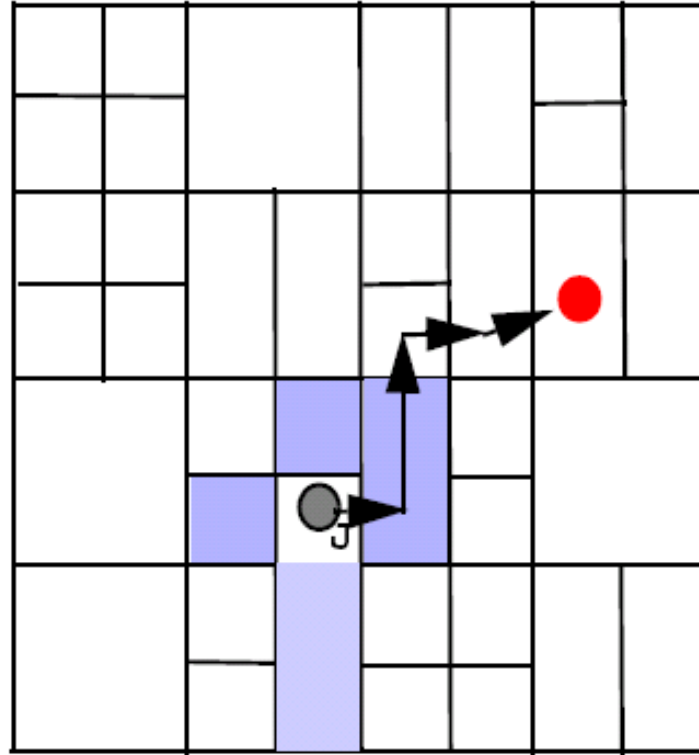
Datensuche / Routing von Nachrichten

- Jeder Knoten kennt nur die unmittelbaren Nachbarn seines Teils des Schlüsselraums
 - D.h., die Routinginformation pro Knoten ist abhängig von der Dimensionszahl und den Aufteilungen (siehe später), aber unabhängig von Gesamtknotenzahl
- Nachrichten enthalten als Zieladresse den Zielpunkt.
- Ein Knoten leitet eine Nachricht zu dem Nachbarknoten weiter, der den geringsten euklidischen Abstand zum Zielpunkt hat.
- Die mittlere Pfadlänge beträgt bei N Knoten und d Dimensionen: $O(N^{1/d})$
 - Pfadlänge wird durch höhere Dimensionen stark reduziert.



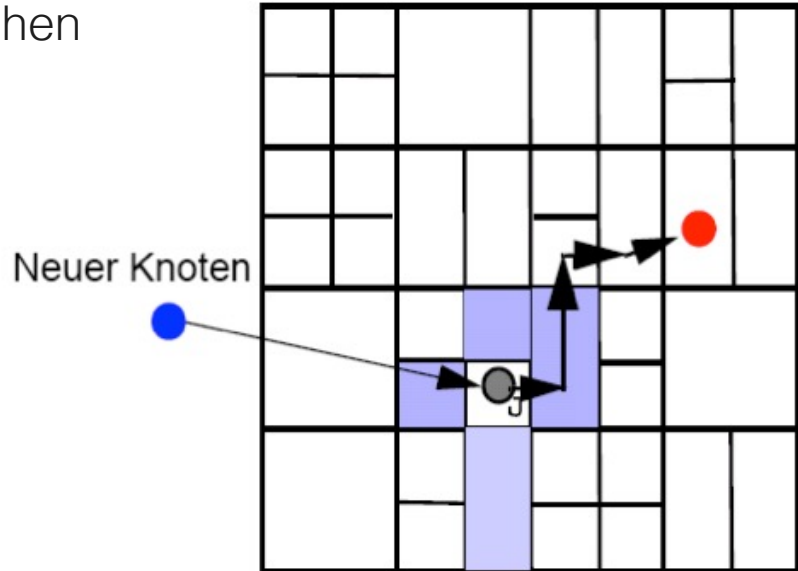
Datensuche / Routing von Nachrichten

- Beispiel



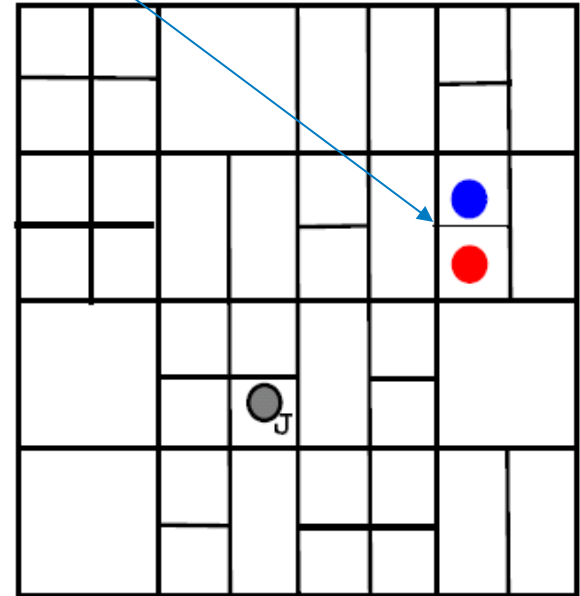
Beitritt eines Knotens: (1) zufälligen Zielbereich wählen

- **Beitretender Knoten** muss einen Knoten K_J kennen
- **Neuer Knoten erzeugt einen zufälligen Punkt** im Koordinatenraum und sendet eine Join-Nachricht zusammen mit seiner IP-Adresse an K_J
- K_J leitet die Join-Nachricht entsprechend des Routingalgorithmus an den zuständigen Knoten weiter.



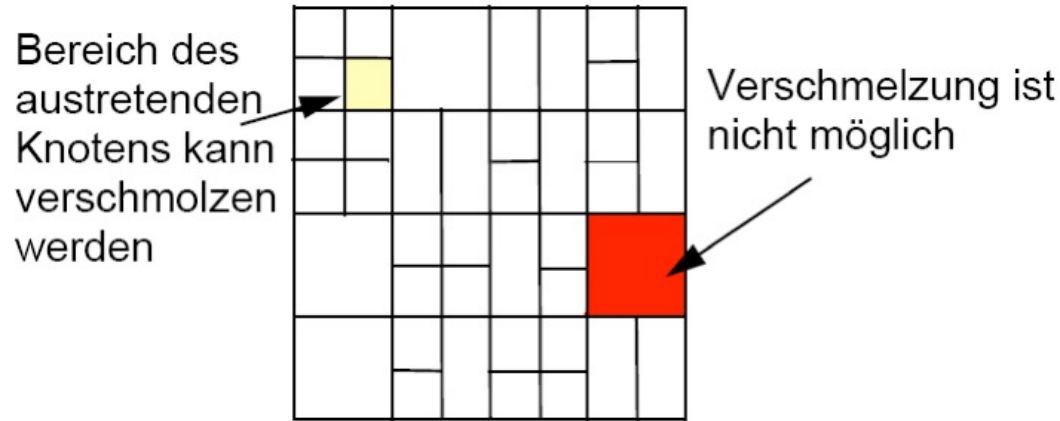
Beitritt eines Knotens: (2) Aufteilung der Zuständigkeiten

- Der für den Abschnitt des Koordinatenraums zuständige Knoten beginnt mit der **Aufteilung (Halbierung) des Koordinatenraums**.
- Entsprechend der Teilung werden verwaltete Schlüssel und Informationen über die Knoten der Nachbarn an den neuen Knoten übertragen.
 - Neuer Knoten wird automatisch bekannt.
- Bem.: Optional die Einfügeposition zu einem Nachbar verschieben, der einen größeren Bereich verwaltet → Volume-Balancing



Abmelden eines Knotens

- Kann ein Knoten nicht mehr am Netzwerk teilnehmen, so übernimmt ein anderer Knoten aus der Nachbarschaft seinen Teil des Koordinatenraums.
- Dabei gibt es zwei Möglichkeiten:
 - (1) Verschmelzung mit einem Bereich, falls möglich.
 - (2) Ist dies nicht möglich, verwaltet der Knoten mit dem kleinsten Bereich den freiwerdenden Bereich mit



Erkennen von Knotenausfällen

- Jeder Knoten sendet regelmäßig Nachrichten an alle seine Nachbarn mit einer Liste seiner Nachbarn, seinem Koordinatenraum und einer Liste der Koordinatenräume seiner Nachbarn (Update Nachricht).
- Benachbarte Knoten, deren Nachrichten (definierter Timeout) ausbleiben, werden als ausgefallen betrachtet.



Erkennen von Knotenausfällen: TAKE-OVER

- Hat ein Knoten K_i erkannt, dass sein Nachbar K_f ausgefallen ist, startet er einen Timer.
- Jeder der Nachbarn von K_f kann dies tun
- **Laufzeit des Timers variiert in Abhängigkeit des von K_i verwalteten Volumens.**
 - Timer umso größer, je größer das Volumen ist
 - Konsequenz: der Timer läuft zuerst bei Knoten mit kleinem Volumen ab



Erkennen von Knotenausfällen: TAKE-OVER

- **Läuft bei einem der Knoten K_i der Timer ab**, sendet er an alle Nachbarn des ausgefallen Knotens eine Takeover-Nachricht.
- Damit erfahren andere Nachbarn von K_f , dass K_i den Koordinatenraum übernehmen will.
- Bei Empfang der Takeover-Nachricht:
 - Beenden andere Nachbarn ihr Bemühen den Koordinatenraum zu übernehmen
 - Oder, falls sie einen kleineren Bereich als K_i verwalten melden Sie dies an K_i und übernehmen den Koordinatenraum.



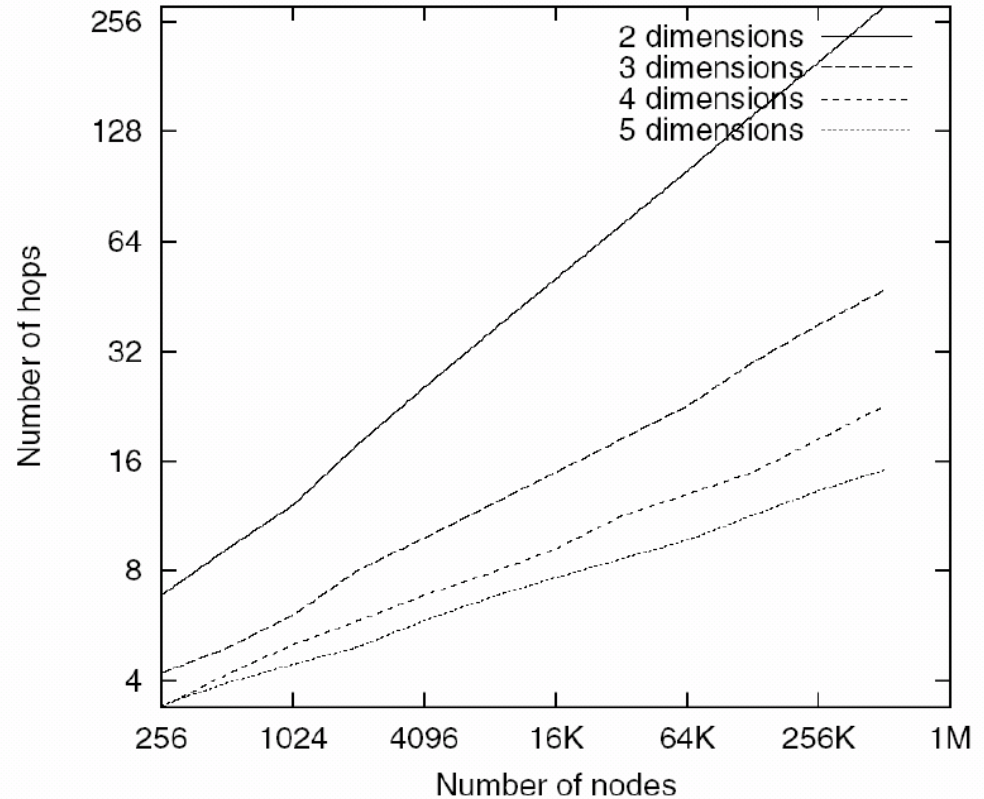
Gleichzeitiger Ausfall mehrerer Nachbarn

- Stellt ein Knoten fest, dass nicht nur ein benachbarter Knoten ausgefallen ist, sondern zusätzlich mehr als die Hälfte der Nachbarn des ausgefallenen Knotens, kann er nicht mehr einfach eine Übernahme des Koordinatenraums vornehmen.
- **Der Knoten muss stattdessen über eine erweiterte Suche zusätzliche angrenzende Knoten finden.**
 - Erst dann kann der Koordinatenraum übernommen werden.



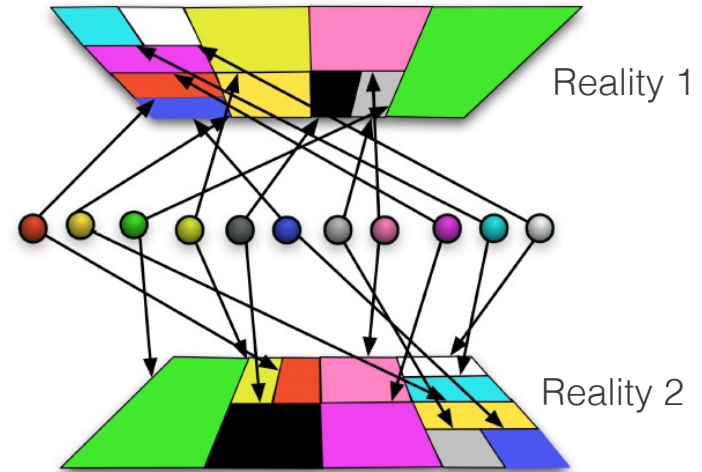
Optimierung: mehr Dimensionen

- Verringert die durchschnittliche Pfadlänge
- Aber erhöhter Aufwand für die Erhaltung der Netzstruktur, da jeder Knoten mehr Nachbarn hat

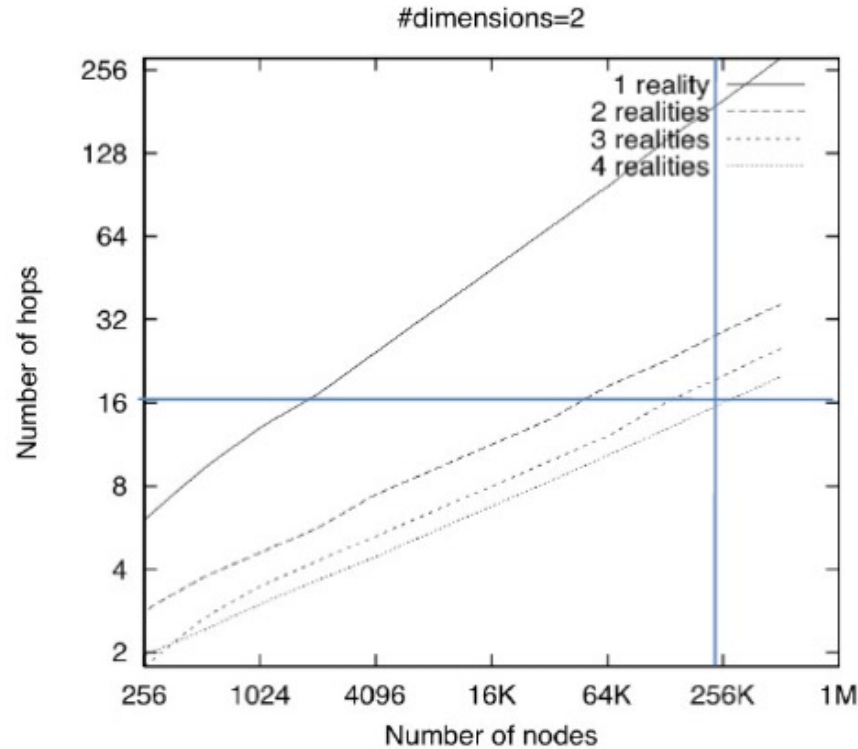


Optimierung: mehrere Koordinatenräume (realities)

- Verringert ebenfalls die durchschnittliche Pfadlänge
- **Routing mit Hilfe verschied. Realities** (unterschiedliche Hash-Fkt.)
 - Je nachdem wo die kürzeste Entfernung vorliegt
 - Erhöht auch die Zuverlässigkeit → Routing um ausgefallene Knoten möglich
 - Im Prinzip mehrere CAN-Netzwerke parallel
 - Damit entsprechend mehr Netzwerkverkehr



Optimierung: mehrere Koordinatenräume (realities)



Mehrere Knoten pro Teilbereich des CAN-Raums

- Ziel: Robustheit gegen Peer-Ausfälle verbessern
- Lösung: Jeder Teilbereich kann von MAXPEERS besetzt werden
- Aufteilen einer Zone erst, wenn MAXPEERS überschritten wird
- Peers für einen Teilbereich kennen sich alle, aber jeder davon kennt nur einen Nachbarn pro angrenzenden Bereich
- Aber erhöhte Komplexität.



Optimierung der Lastverteilung durch Caching

- Jeder Knoten verfügt über einen lokalen Cache.
- **Cache-Einträge haben TTL**
- Sind Daten im Cache vorhanden, so werden damit Anfragen beantwortet.
- Je populärer Daten sind, desto häufiger werden sie angefragt und desto häufiger sind sie in vielen Caches vorhanden



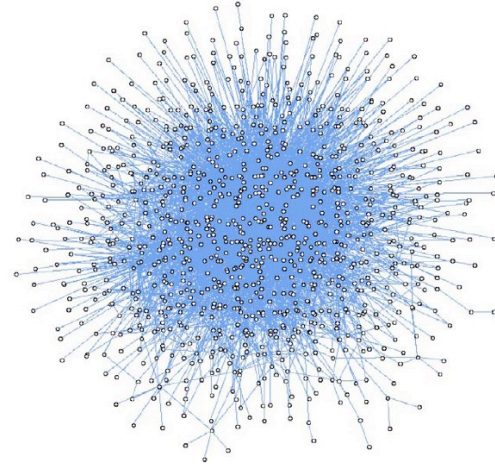
- Vorteile
 - Findet Daten, sofern vorhanden
 - Balanciert die Datenmenge pro Knoten
 - Kennt verschiedene Wege zum Ziel und kann dadurch beste Route wählen
 - Eine Reihe von Optimierungen helfen das Routing zu beschleunigen und die Robustheit zu verbessern
- Nachteile:
 - U.U. lange Routingwege, je nach Aufteilungsmuster (deswegen Reorganisation im Hintergrund; siehe Publikation)
 - Stabilität durch geringe Nachbarzahl gefährdet (abhängig von Dimension)

11.4 Unstrukturierte Peer-to-Peer Systeme



Gnutella

- Arbeitet völlig ohne zentrale Strukturen (2000)
- Erstes vollständig dezentrale P2P-System
- Protokoll zur verteilten Suche nach Dateien.
- Alle Peers sind mit einer konfigurierbaren Anzahl an Nachbarn (bekannte Peers) verbunden
- „Netzwerkstruktur“ →



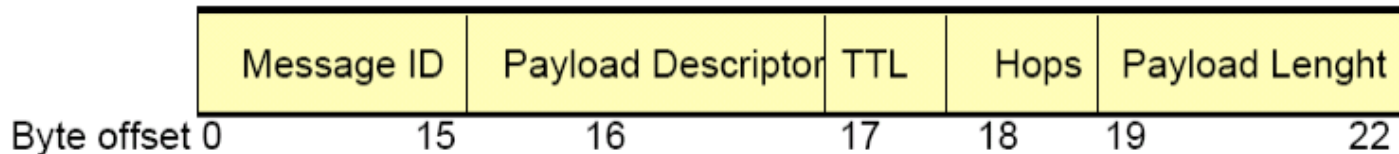
Knotenbeitritt in Gnutella 0.4

- Zur Teilnahme benötigt ein Peer mindestens eine IP-Adresse eines bereits teilnehmenden Knotens.
- Problem: Es gibt keinen festen zentralen Server
- Lösung: Installation enthält eine Liste „stabiler“ Peers
 - Liste wird beim Beitritt durchprobiert
 - Von dem gefundenen Peer aus wird das Netz erkundet
 - Dabei wird die Liste bekannter Peers aktualisiert, erweitert und abgespeichert



Gnutella - Nachrichtenformat

- Message ID: besteht z. B. aus Zeitstempel, Zufallszahl und MAC-Adresse
- Payload Descriptor: bestimmt den Nachrichtentyp.
- Time to Live (TTL): Begrenzung der Suchtiefe (Default = 7)
 - Wird bei jedem Schritt um eins verringert
 - Nachricht wird bei TTL=0 verworfen
- Hops: bestimmt, wie oft die Nachricht bereits weiter geleitet wurde (dient Gegenkontrolle, dass TTL nicht größer als 7 war)



Ping-Nachricht

- Ziel: neue aktive Peers finden
 - Und auch als Keep-Alive-Nachricht
 - Wird von jedem Peer periodisch versendet
- Ping-Nachricht wird im P2P-Netzwerk geflutet
 - Eingehende Nachrichten werden an alle Nachbarn weitergeleitet, außer an den, von dem das Paket empfangen wurde
 - Empfänger merkt sich Message-ID und Nachbar, von dem die Ping-Nachricht empfangen wurde (für Rückwärts-Routing der Antwort)
 - Später eintreffende Duplikate werden verworfen (erkennbar anhand einer Message-ID)



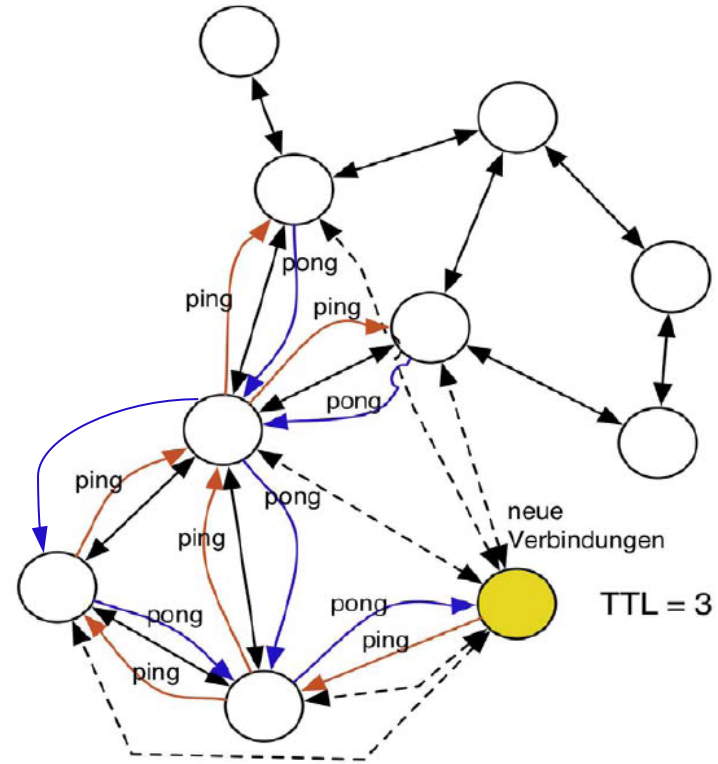
Pong-Nachricht

- Pong ist die Antwort auf einen Ping.
- Schickt jeder Peer der einen Ping empfängt
- Enthält ID der Ping-Nachricht (Duplikate erkennbar).
- Und als Payload Informationen über den antwortenden Knoten:
 - IP-Adresse und Port des Rechners
 - Anzahl der bereitgestellten Dateien
 - Größe der bereitgestellten Daten
- Wird auf umgekehrten Weg zurückübertragen.



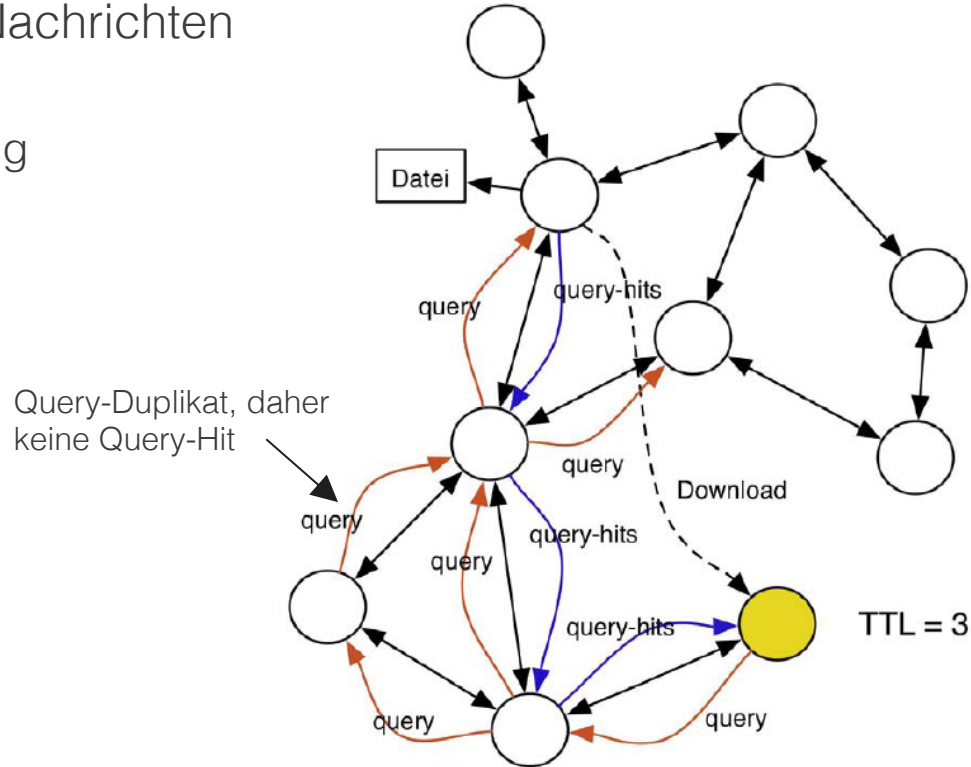
Knotenbeitritt: Gnutella 0.4

- Der gelbe Peer tritt bei
 - Nach dem Eintreffen der Pongs sind ihm viele Peers bekannt
 - Er wählt N (typ. N=5) daraus zufällig aus und baut TCP-Verbindungen zu diesen Peers auf. Dies sind dann seine Nachbarn.



Suche / Transfer

- Die Suche verwendet die Nachrichten Query und Query-Hits
 - Funktioniert wie Ping/Pong
- Das eigentliche Laden von Dateien erfolgt direkt zw. zwei Peers.



- TTL führt implizit zu einer Netzpartitionierung:
 - Daten werden u.U. nicht gefunden, obwohl diese vorhanden sind
 - Bei 50.000 Peers konnten 95% aller Peers Nachrichten mit weniger als 7 Hops austauschen
- Aber sehr großer Overhead durch das Fluten von Nachrichten
→ Gnutella ist sehr bald zusammengebrochen

FastTrack (Kazaa) und Gnutella (V. 0.6)

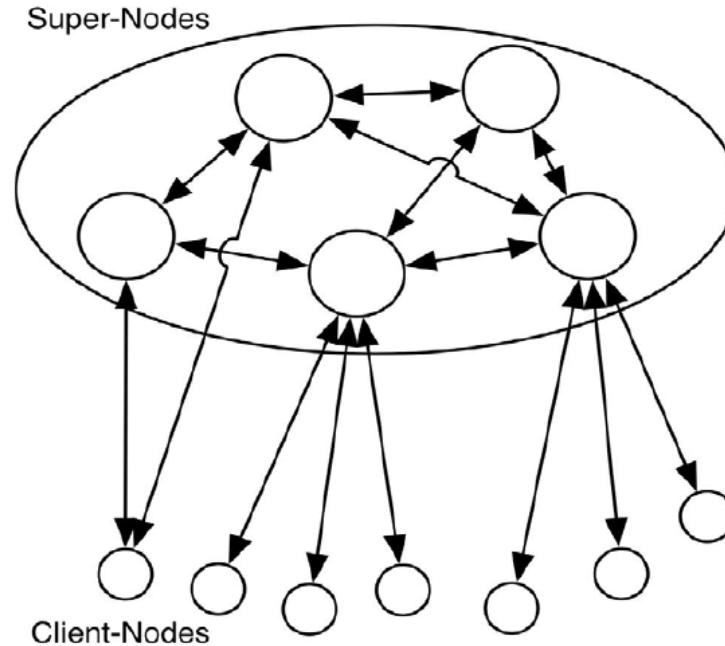
- Einführung von Super-Nodes / Ultra-Peers



- „Einfacher“ Peer ist nur mit einem Super-Node verbunden und teilt diesem seine IP-Adresse und Dateiliste mit.
- Super-Nodes agieren als Proxy für einfache Peers:
 - Unterhalten einer P2P-Netz im Stil von Gnutella.
 - Beantworten Anfragen von Peers.
 - Peers werden so entlastet.
 - Super-Nodes werden je nach verfügbarer Bandbreite und CPU-Leistung dynamisch bestimmt.

FastTrack (Kazaa) und Gnutella (V. 0.6)

- Einführung von Super-Nodes / Ultra-Peers



11.4 Zusammenfassung



Zusammenfassung

- P2P-Systeme sind dezentral, selbstorganisierend und für sehr viele Peers im Internet konzipiert.
- Sie verwenden Eventual Consistency
- Unstrukturierte P2P-Systeme müssen keine Struktur aufrecht erhalten, finden aber u.U. Daten nicht, auch nicht im fehlerfreien Fall, obwohl sie vorhanden sind
- Strukturierte P2P-Systeme finden vorhandene Daten im fehlerfreien Fall immer, müssen aber die Struktur im Fehlerfall reparieren



Zusammenfassung

- Einige Konzepte der P2P-Systeme wurde/werden in Cloud- und BigData-Systemen übernommen.
- Consistent Hashing: elegante Lösung zur Daten- und damit Lastverteilung
- Replikation: bessere Verfügbarkeit und einfaches Recovery
- Inkrementelle Skalierbarkeit: neue Server dynamisch hinzufügen/entfernen
- Selbstorganisation: benötigt weniger Konfigurationsarbeit
- Aber i.d.R. kennen sich alle Server gegenseitig, womit die Suchzeit $O(1)$ ist.

